STUDENT GUIDE · VIT VELLORE GUEST LECTURE

Build your first AI app with Java

Set up in your browser, run the Campus Assistant, and change it yourself. You install nothing.

What you will do

- Open a coding computer inside your browser (GitHub Codespaces).
- Run a small AI app that chats, books lab slots, and answers from a college handbook.
- Change its personality, its data, and its abilities with a few lines of code.

What you need

- A laptop with Chrome or Edge
- A free GitHub account (we make one in Step 1)
- An email you can open right now
- About 45 minutes

You do not need

- To install Java or anything else
- An API key or a paid AI plan
- To know Java well. Every step says what to click and what to paste.

The steps

1	Create a GitHub account	**6**	Start the app
2	Open the project	**7**	Send your first question (be patient, 3 minutes)
3	Create your Codespace	**8**	Try the demos
4	Trust the folder and wait	**9**	Do the exercises (and extra practice, 5 to 10)
5	Find your way around	**10**	If something goes wrong, and how to stop

Project: https://github.com/usv240/spring-ai-campus-assistant · Playground (no setup): https://usv240.github.io/spring-ai-campus-assistant/

1 Create a GitHub account

GitHub is a website where programmers keep their code. Codespaces is a free feature of GitHub. Already have an account? Skip to Step 2.

1. Go to **github.com/signup**.
2. Enter your email, a password, and a username. Pick a username you won't mind showing to employers one day.
3. Solve the puzzle, then open your email.
4. GitHub sends you a **verification code**. Type it in. This is the **verify your email** step. Without it Codespaces will not start.
5. You can skip any questions about plans, teams, or surveys. Choose the **Free** plan.

Do this before the session. The verification email can take a few minutes. Check your spam folder if it does not arrive.

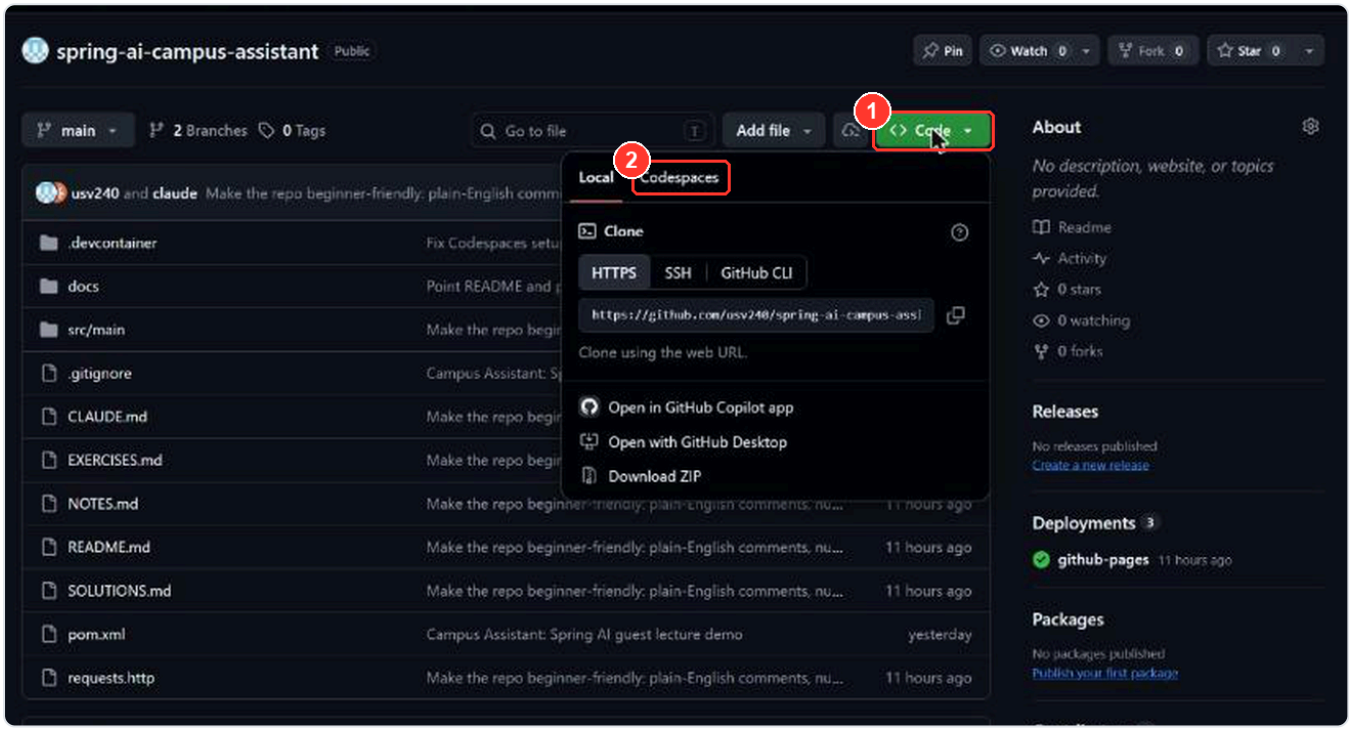
Is it really free? Yes. A free GitHub account includes about 120 core-hours of Codespaces a month. Our Codespace uses 4 cores, so that is roughly **30 hours** of use, far more than this session needs. Remember to **stop** your Codespace when you finish (Step 10) so you don't waste hours.

Student Developer Pack: if you have a student email, you can also apply at education.github.com/pack for extra hours. It is optional.

2 Open the project

The project is a folder of code that I put on GitHub. We will open a copy of it in your browser.

1. Sign in to GitHub.
2. Open this address in your browser:
   github.com/usv240/spring-ai-campus-assistant
3. Find the green button that says **Code** (marked 1 below) and click it.
4. In the box that opens, click the **Codespaces** tab (marked 2).



The repo page. 1 = the green Code button. 2 = the Codespaces tab.

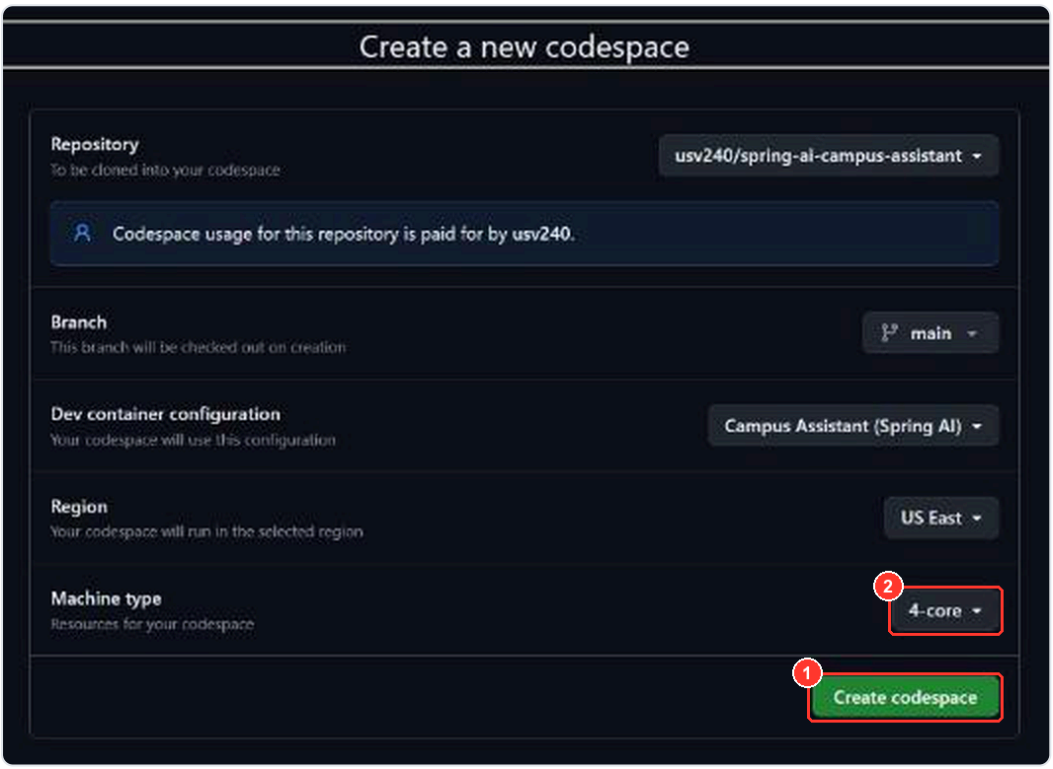
> **Shortcut:** scroll down the project page to the README and click **Open in GitHub Codespaces**. It takes you straight to Step 3.

On your screen the Codespaces tab shows a green **Create codespace on main** button. In the picture it shows the author's own codespaces instead, because the author already has some.

3 Create your Codespace

A Codespace is a computer that lives in the cloud and shows up as a coding window in your browser. GitHub sets it up for you, including Java and the AI model.

1. Click **Create codespace on main**. You land on the page below.
2. Check that **Machine type** says **4-core** (marked 2). That is the one we want. If it says 2-core, change it to 4-core.
3. Click the green **Create codespace** button (marked 1).
4. A loading screen appears. **Don't close the tab.** It takes about **5 minutes** the first time.



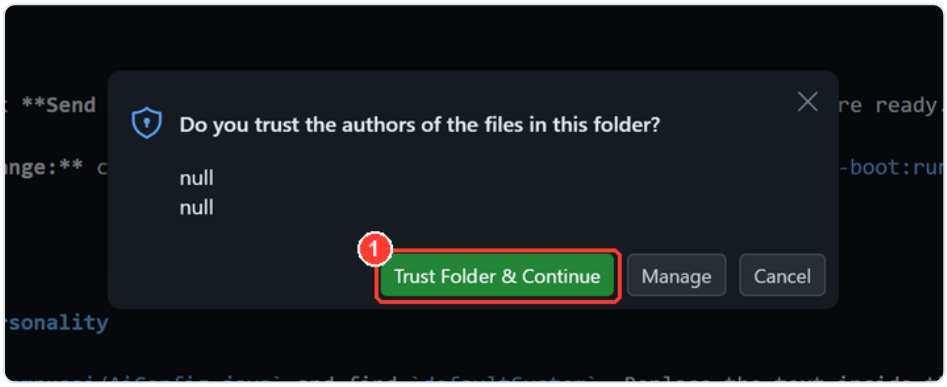
1 = Create codespace. 2 = Machine type, 4-core.

> **While you wait:** the setup is installing the AI model (about 2.3 GB). That is why it takes a few minutes. You can read Step 5 while you wait.

4 Trust the folder and wait for "All set"

When the coding window appears, it may ask whether you trust the project. Say yes, or the terminal won't work.

1. If a box asks **"Do you trust the authors of the files in this folder?"**, click the green **Trust Folder & Continue** button (marked 1).
2. Look at the bottom of the window. This black area is the **terminal**, where the computer shows messages.
3. Wait until the terminal says **All set**. This means Java, Maven and the AI models are installed.



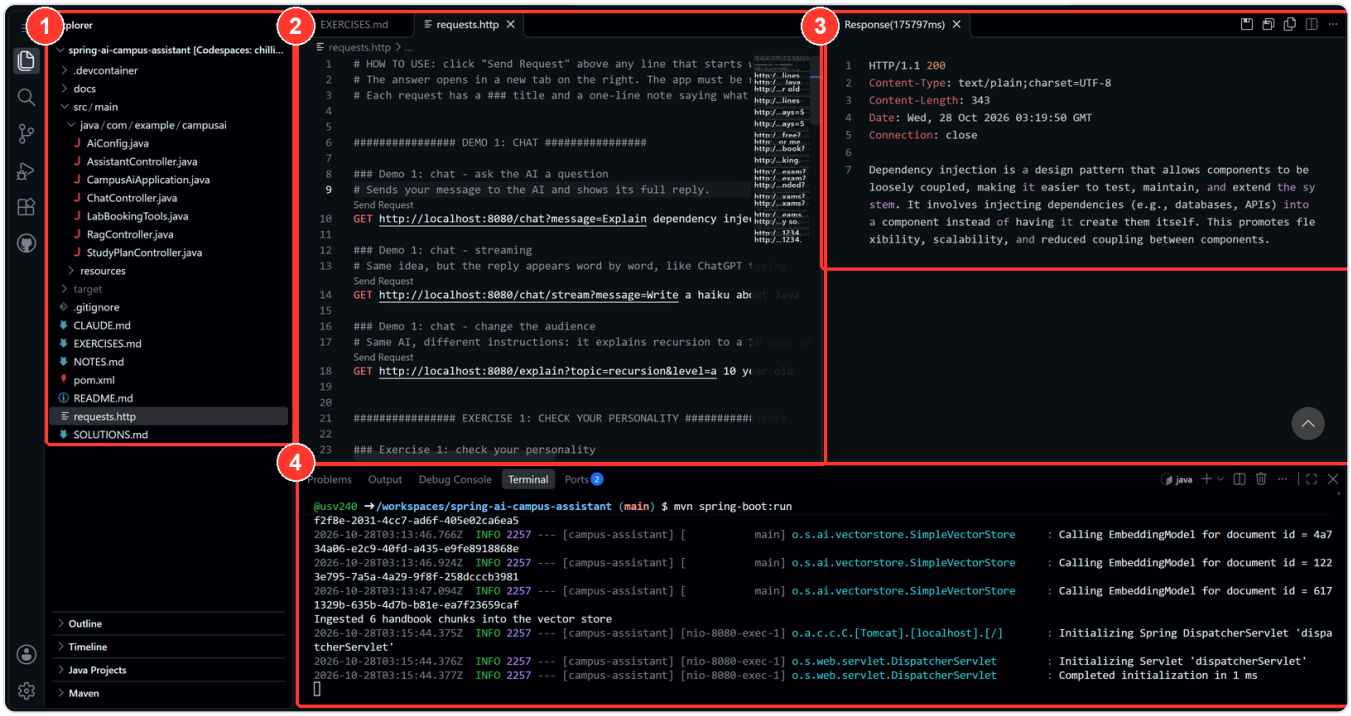
1 = Trust Folder & Continue.

Not working?

You see	Do this
A yellow bar says **Restricted Mode**	Click it and choose **Trust**.
The terminal is empty	Wait a minute. If it stays empty, refresh the page.
No "All set" after 8 minutes	Refresh the page. If it is still stuck, tell the host in the chat. You can watch and try later.

5 Find your way around

Your Codespace looks like a code editor called VS Code. It has four areas you will use.



1 = Files. 2 = Editor. 3 = Answer panel. 4 = Terminal.

1. Files	The list of files in the project. Click one to open it.
2. Editor	Where you read and change code. Press **Ctrl+S** to save.
3. Answer panel	When you send a question to the app, the AI's answer opens here in a new tab.
4. Terminal	Where you start the app and where it prints messages.

> **Ignore the Chat panel** on the right side of your screen. It is a different tool and you don't need it.

The files you will touch

`requests.http`	All the questions you can send. Click a link to send one.
`EXERCISES.md`	The steps for the four exercises.
`AiConfig.java`, `StudyPlanController.java`, `LabBookingTools.java`	Code you change in the exercises. All are in `src/main/java/com/example/campusai`.
`course-handbook.md`	The made-up college handbook the AI reads. In `src/main/resources/docs`.

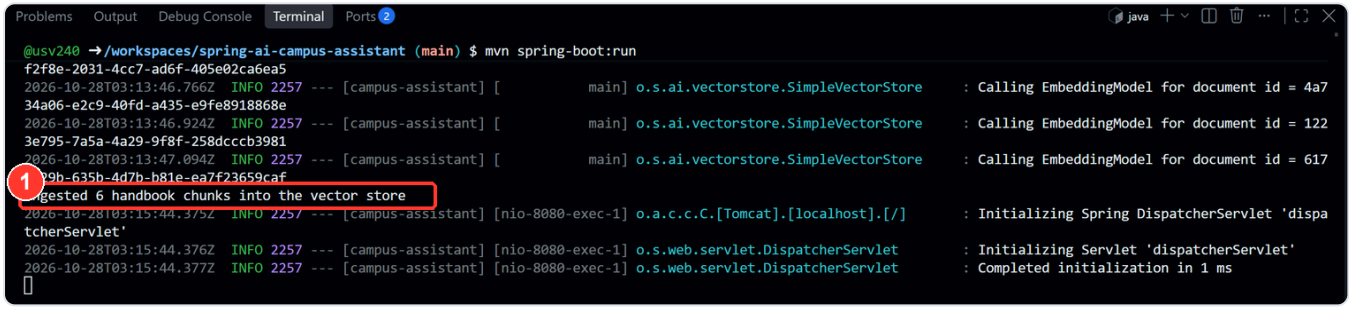
6 Start the app

The app is our Campus Assistant. Starting it turns it on, like opening a shop.

1. Click once inside the **terminal** at the bottom.
2. Type this exactly, then press **Enter**:

   ```
   mvn spring-boot:run
   ```

   (`mvn` is a tool that builds and runs Java apps.)
3. Wait about a minute while text scrolls by.
4. Look for the line **Ingested 6 handbook chunks into the vector store** (marked 1). Then **Started CampusAiApplication**. The app is now running.



1 = the app has read the handbook and is ready.

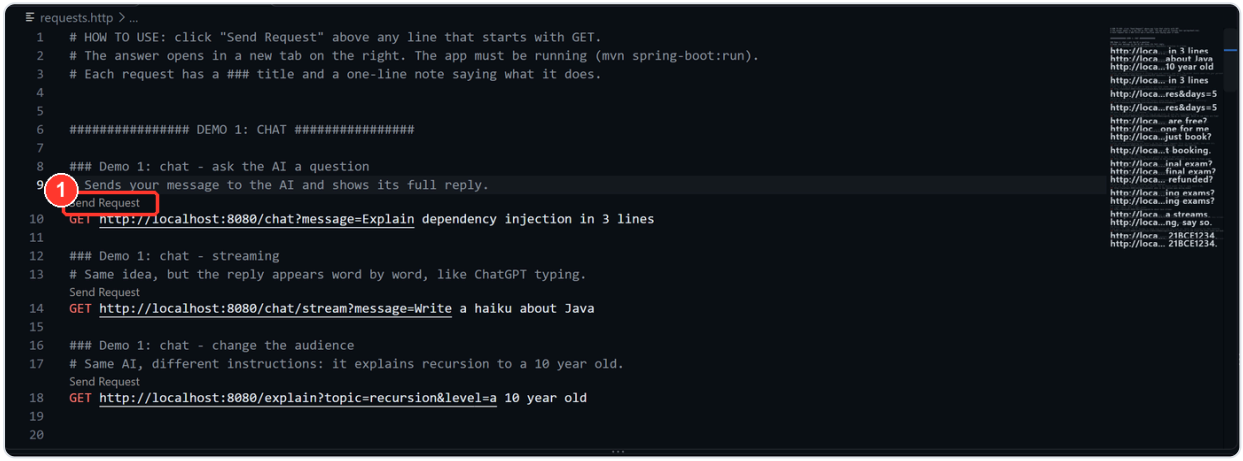
> **Leave this terminal alone.** The app runs inside it. If you close it or press Ctrl+C, the app stops. To start it again, click the terminal, press the **Up arrow**, then **Enter**.

> **Red error text?** Don't panic. Press **Ctrl+C**, then Up arrow, then Enter to try again. If you see `Connection refused`, type `bash .devcontainer/start-ollama.sh` and press Enter, then start the app again.

7 Send your first question

We talk to the app by sending it a web address, like opening a link. A file called `requests.http` has all the links ready.

1. In the file list, click **`requests.http`**.
2. Find the first line that starts with **GET**. Just above it you will see small gray words **Send Request** (marked 1).
3. Click **Send Request one time only**.
4. Look at the bottom bar. It says **Waiting...** (marked below). That means it worked, and the AI is thinking.



1 = Send Request, above the first GET line.



1 = the "Waiting..." message in the bottom bar.

> **The very first answer takes about 3 minutes.** The AI model is loading into memory. That is normal. **Do not click again**, because that only makes the wait longer. After this first answer, replies take 5 to 30 seconds.

7 Read the answer

When the AI finishes, a new tab opens on the right with the reply.



1 = the tab title shows how long it took (in milliseconds). 2 = `HTTP/1.1 200` means success. The AI's text is below it.

> **You did it.** You just talked to an AI model from your own Java app, running in your own Codespace.

What did that line mean?

```
GET http://localhost:8080/chat?message=Explain dependency injection in 3 lines
```

`GET`	"Please give me something." It is what your browser does when you open a web page.
`localhost`	"This same computer." Here that means your Codespace.
`8080`	The door the app listens on.
`/chat`	The feature we want: chat with the AI.
`?message=...`	What we are sending: our question.

> **Got an error page (HTTP 500)?** Small AI models slip sometimes. Click **Send Request** again.

8 Try the demos

Each demo adds one idea. In `requests.http`, click Send Request on each one in order, and read the code file next to it. Answers can take up to 35 seconds in a Codespace. Click once, then wait.

Demo	The problem	What to send, and what to look for
1. Chat ChatController.java	We want our own app to answer a student's question instantly.	**chat**: a normal reply. **streaming**: the reply appears word by word. **change the audience**: the same AI explains for a 10 year old.
2. Neat answers StudyPlanController.java	A paragraph of text can't fill a table. Apps need data in a clear shape.	**study plan**: you get days, topics and hours as neat data (JSON), not a paragraph.
3. Memory and tools AssistantController.java LabBookingTools.java	An AI forgets you after every message, and it can only talk. It can't book a lab.	Send **message 1**, then **message 2** ("book the 2 PM one"). It remembers! Then look in the **terminal** for `>>> TOOL EXECUTED: booked AI Lab at 14:00`. That line proves Java really ran. Then send **a different student**: it has no memory of the booking.
4. Your own documents RagController.java AiConfig.java	The AI has never read our handbook, so it guesses rules like attendance. A wrong guess can cost a student an exam.	**without the handbook**: it guesses or says "I don't know". **with the handbook**: it says **82%**. **another question**: re-evaluation costs **750 rupees**.

Words to remember. **Tool calling** = the AI asks our Java code to do something. **RAG** = we give the AI the right page from our own documents before it answers, like an open-book exam.

Booking demo: a slot can only be booked once. If you test it twice and see "slot not available", restart the app (Step 6) to reset the slots.

9 Do the exercises

Four small changes. For each one: open the file, find the **EXERCISE START HERE** comment, paste the code, save, restart, then send the check request. Full steps are in `EXERCISES.md`. Finished code is in `SOLUTIONS.md`.

How to restart after a change

Click in the terminal, press **Ctrl+C**, press the **Up arrow**, press **Enter**. It takes about 20 seconds. Always **save** the file first with **Ctrl+S**.

Exercise 1: Give the AI a personality (5 min)

File: `AiConfig.java`, look for `// EXERCISE 1 START HERE`. Replace `.defaultSystem("""...""")` with:

```
.defaultSystem("""
        You are Campus Assistant. Answer like an excited cricket commentator,
        in under 60 words.
        """)
```

Check: send "Exercise 1: check your personality". The answer should sound like a commentator.

Exercise 2: Add a "difficulty" field (5 min)

File: `StudyPlanController.java`, look for `// EXERCISE 2 START HERE`. Change the `StudyTask` line to:

```
public record StudyTask(String day, String topic, int hours, String resource, String
difficulty) {}
```

Check: send "Exercise 2: check the difficulty field". Every task now has a `"difficulty"`. If you get an error page, send it again.

9 Exercises, continued (try these on your own)

Exercise 3: Write a cancelBooking tool (10 min)

File: `LabBookingTools.java`, scroll to the bottom and find `// EXERCISE 3 START HERE`. Paste this **below that line and above the last** `}`:

```
@Tool(description = "Cancel a lab booking and make the slot free again")
public String cancelBooking(@ToolParam(description = "Lab name, e.g. AI Lab") String lab,
                            @ToolParam(description = "Time slot in HH:mm") String time) {
    lab = findLab(lab);
    time = toHHmm(time);
    List<String> slots = freeSlots.get(lab);
    if (slots == null) {
        return "Cancel failed: there is no lab called " + lab + ".";
    }
    if (!slots.contains(time)) {
        slots.add(time);
    }
    System.out.println(">>> TOOL EXECUTED: cancelled " + lab + " at " + time);
    return "Cancelled " + lab + " at " + time + ". The slot is free again.";
}
```

Check: restart, book 14:00 with the two Demo 3 requests, then send "Exercise 3: cancel the booking". Look in the terminal for `TOOL EXECUTED: cancelled AI Lab at 14:00`.

Exercise 4: Add a Canteen section to the handbook (5 min)

File: `src/main/resources/docs/course-handbook.md`. At the very bottom, find `<!-- EXERCISE 4: add your new section below this line -->` and paste below it:

```
## Canteen
The main canteen is open from 7 AM to 10 PM every day. The night canteen near
the hostels stays open until 2 AM during exam weeks.
```

Check: restart (the handbook is read only at startup), then send the two "Exercise 4" questions. With the handbook, it says **2 AM**. Without it, it guesses.

Bonus: give the AI a goal with no steps, like "Find me a free slot in the Cloud Lab after lunch and book it." Watch the **terminal** for `TOOL EXECUTED`. Small models sometimes say "booked" when nothing happened, so trust the terminal, not the chat reply.

Stuck on an exercise? The most common mistake is a missing comma, bracket or quote. Read the red line in the terminal, which names the problem. Compare your file with `SOLUTIONS.md`.

+ Extra practice: Exercises 5 to 10

Finished Exercises 1 to 4? Try these. Full steps with the code to paste are in `EXERCISES.md` (Parts C and D). Finished code is in `SOLUTIONS.md`. Each one has a matching request in `requests.http` under "EXTRA PRACTICE".

Part C: easy (5 to 8 minutes each)

Exercise	Where	What you should see
5. Change the temperature	`application.yml`, `temperature: 0.3`	Send the slogan request 3 times at **0.1**: the same slogan. At **1.2**: different slogans. Set it back to 0.3.
6. Add a new lab	`LabBookingTools.java` (2 places) and `AssistantController.java`	The assistant lists the Robotics Lab slots (12:00, 17:00) and books one. The terminal shows `TOOL EXECUTED`.
7. Watch the AI choose a tool	`application.yml`, `org.springframework.ai: DEBUG`	Send the two Demo 3 requests, then search the terminal (Ctrl+F) for `Executing tool call`. Set it back to `INFO`.

Part D: challenges, a little harder (10 minutes each)

Exercise	Where	What you should see
8. Make a quiz maker	New file `QuizController.java`	`/quiz?topic=Java streams&count=3` returns 3 questions, each with 4 options and an answer.
9. A second document	New file `hostel-rules.md`, and the `ingestHandbook` method in `AiConfig.java`	The startup line shows more chunks (7 instead of 6). Asking about hostel gates gives **10 PM**. The attendance answer is still 82%.
10. Let Java say no	`LabBookingTools.java`, inside `bookSlot`	A booking with reg no `hello` is refused and the terminal shows no `TOOL EXECUTED`. A valid one like `22BIT0042` books.

> **Always:** save the file (Ctrl+S) and restart the app after each change (terminal, Ctrl+C, Up arrow, Enter). Small AI models sometimes slip, so if you get an error page, send the request again.

> **Idea for your own:** an exercise 10 stretch: allow each student at most 2 bookings, and refuse the third. Keep a count per registration number in Java.

10 If something goes wrong, and how to stop

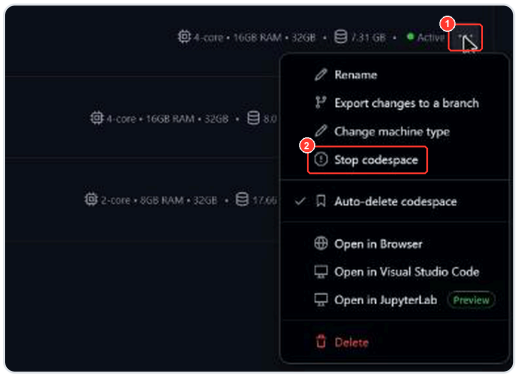
Quick fixes

What happens	What to do
I clicked Send Request and nothing happened	Look at the bottom bar. If it says **Waiting...**, it worked. The first answer takes about 3 minutes. Don't click again.
Error page, HTTP 500	Click Send Request again. Small AI models slip sometimes.
`Connection refused`	Type `bash .devcontainer/start-ollama.sh` in the terminal, press Enter, then start the app again.
The app won't start after my change	Read the red line in the terminal. It names the file and line. Fix the typo, or compare with `SOLUTIONS.md`.
I changed something and nothing is different	Did you save (Ctrl+S) **and** restart the app?
The AI says "booked" but nothing happened	Check the terminal for `TOOL EXECUTED`. The terminal is the truth. The AI can be wrong.

When you are done: stop your Codespace

A Codespace keeps using your free hours while it is running. Stop it when you finish. Your files stay saved.

1. Go to **github.com/codespaces**.
2. Click the three dots **...** on the right of your Codespace (marked 1).
3. Click **Stop codespace** (marked 2).



1 = the three dots. 2 = Stop codespace.

> **Come back later:** click your Codespace's name on that page to restart it, then run `mvn spring-boot:run`. The first answer takes a few minutes again.

Keep learning: read `NOTES.md` in the project (every idea in simple words, with a glossary). Then build one small project end to end, put it on GitHub, and show it. That is how you get noticed.